Logic Documentation

1. How the Chatbot Processes User Queries

When a user submits a query via the Streamlit frontend, the query enters a sequential pipeline designed for speed and safety:

1. **Input Sanitization**: The query is stripped of excess whitespace and normalized.
2. **Pre-generation Safety Check**: The query is intercepted by the `SafetyGuardrail` system. Static regex patterns check for acute emergency keywords (e.g., "heart attack", "suicide"). If triggered, processing halts immediately, and a hardcoded emergency protocol is returned.
3. **Retrieval (RAG)**: If safe, the query is passed to the embedding model (`all-MiniLM-L6-v2`) to generate a vector. This vector is searched against the FAISS index to retrieve the top-K most relevant document chunks from the predefined healthcare PDFs and any user-uploaded PDFs.
4. **Assembly**: The retrieved chunks, the user's health profile, and the recent conversation history are passed to the LangChain orchestrator.

2. How Prompts are Customized

Prompts are highly dynamic and tailored to the exact context of the user and their query:

- **System Prompt**: Defines the persona ("You are a Healthcare AI Assistant...") and sets explicit boundaries (never diagnose, never prescribe).
- **Profile Injection**: If the user has populated their Health Profile in the sidebar (e.g., Age, Allergies, Known Conditions), these are injected directly into the system prompt. The LLM is instructed to use this information to personalize advice (e.g., considering a peanut allergy when discussing nutrition) without crossing the line into medical diagnosis.
- **RAG Context**: Retrieved document chunks are formatted as simulated user-assistant exchanges. This "few-shot" grounding ensures the LLM heavily weights the retrieved medical literature over its general pre-training data.

3. How Responses are Generated

Response generation is handled by the `HealthcareChatbot` class via LangChain:

- The assembled prompt (System Prompt + History + Context + User Query) is routed to the active LLM Provider (either Google Gemini 2.5 Flash or Groq Llama 3.3).
- The LLM streams the response back token-by-token for a fast, interactive user experience.
- The generation is constrained by a `temperature` of 0.3 to ensure factual consistency and reduce hallucinations.
- **Source Citations**: Post-generation, the UI appends the source documents (PDF names) that were used during the RAG retrieval phase, providing transparency.

4. Safety Measures and Validation

Safety is the most critical component of this architecture, implemented via a multi-layered approach:

- **Layer 1: Emergency Regex Checks**: Hardcoded blocking of life-threatening queries, prioritizing immediate intervention over AI advice.
- **Layer 2: Diagnosis & Prescription Regex**: Blocks explicit attempts to solicit diagnoses or medication dosages.
- **Layer 3: Dynamic LLM Scope Enforcement**: The System Prompt explicitly instructs the model to refuse off-topic questions (e.g., programming, politics) and subtly refuse medical diagnoses by offering general educational information instead.
- **Layer 4: Disclaimer Injections**: Both the UI and the generated responses carry mandatory disclaimers indicating the AI is for educational purposes only.

5. Assumptions Made During Development

- **English-Only Interaction**: The system and safety regex patterns assume interactions are primarily in English.
- **Single-User Environment**: Streamlit's session state is used assuming a single local user per browser tab. Uploaded PDFs are stored in memory and tied to the active session.
- **Predefined Knowledge Validity**: It is assumed that the provided medical documents (`data/knowledge/*.pdf`) contain accurate, validated medical information, as the LLM will treat them as absolute ground truth.
- **Hardware Profile**: FAISS and HuggingFace Embeddings are configured to run on CPU to ensure maximum compatibility across all testing environments without requiring a dedicated GPU.